

Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos
Docente: Ing. Manuel Gustavo Rojas Vásquez



Universidad
Rafael Landívar
Identidad Jesuita en Guatemala

PROYECTO 1

Aplicación de Escritorio con Gestión de Configuración de Usuario

Peláez Virula, Moshé Arenz – 1556425

Quetzaltenango, Septiembre 2026

TABLA DE CONTENIDO

1.	<i>Formato de almacenamiento: JSON.....</i>	3
2.	<i>Escritura segura y respaldo.....</i>	4
3.	<i>Manejo de casos de error.....</i>	5
4.	<i>Ejemplo de archivo de configuración generado</i>	7
5.	<i>Capturas de pantalla del funcionamiento</i>	8
5.1.	Carga inicial (archivo ausente → valores por defecto).....	8
5.2.	Menú principal con Settings funcional	9
5.3.	Selectores nativos: color y foto de perfil.....	10
5.4.	Formulario de Settings completo y guardado exitoso	11

1. Formato de almacenamiento: JSON

Se eligió JSON frente a XML (alternativa descartada) para el archivo de configuración de usuario. Para una configuración plana de pares clave-valor, sin jerarquías profundas ni necesidad de validación por esquema, JSON ofrece menor complejidad de código y menor superficie de error sin sacrificar legibilidad.

Criterio	JSON	XML (descartado)
Tamaño	Más compacto, sin etiquetas de cierre repetidas	Más pesado por las etiquetas de apertura/cierre
Legibilidad	Estructura clave-valor clara para configuraciones planas	Verboso para pares clave-valor simples
Facilidad de parseo	Soporte nativo en Python (módulo json), sin dependencias	Requiere xml.etree o librerías extra, más código
Robustez ante corrupción	Un solo JSONDecodeError cubre casi todos los errores	Errores de parseo más variados y difíciles de anticipar
Tipos de datos	Mapea directo a dict/int/str de Python	Todo es texto; hay que convertir tipos manualmente

2. Escritura segura y respaldo

La función `save_config()` sigue tres pasos para que un cierre abrupto de la aplicación nunca deje el archivo de configuración corrupto:

1. Se respalda la configuración anterior en `config.json.bak` (si existe).
2. Se escribe el contenido nuevo en un archivo temporal `config.json.tmp`.
3. Se reemplaza el archivo final con `os.replace()`, que es atómico a nivel de sistema operativo.

```
# 1. Respaldo de la configuración anterior (si existe)
if os.path.exists(path):
    with open(path, "r", encoding="utf-8") as src, \
        open(backup_path, "w", encoding="utf-8") as dst:
        dst.write(src.read())

# 2. Escritura a archivo temporal
with open(tmp_path, "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=4)

# 3. Reemplazo atómico del archivo final
os.replace(tmp_path, path)
return None
```

`os.replace` es atómico: si la aplicación se cierra durante el paso 2, `config.json` conserva su versión anterior intacta; el archivo nuevo solo reemplaza al viejo una vez que está completamente escrito.

3. Manejo de casos de error

`load_config()` y `save_config()` capturan explícitamente los tres casos, degradando siempre a un comportamiento definido (nunca a un traceback sin capturar):

- **Archivo ausente** → se usan los valores por defecto, sin excepción ni aviso.
- **Archivo corrupto / formato inválido** → se captura `json.JSONDecodeError`, se usan valores por defecto y se avisa al usuario.
- **Falta de permisos de lectura/escritura** → se captura `PermissionError`, degradando a valores por defecto (lectura) o avisando que el guardado falló (escritura).

```
# Caso 1: archivo ausente
if not os.path.exists(path):
    logger.info("Archivo de configuración no encontrado, usando valores por defecto.")
    return DEFAULT_CONFIG.copy(), None

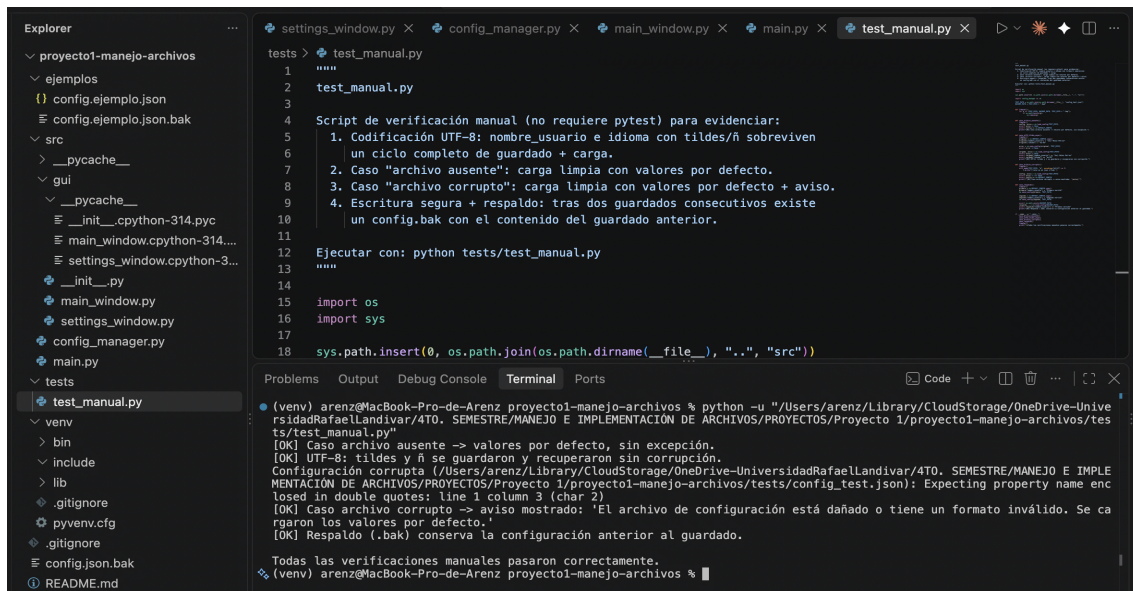
try:
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)

    config = DEFAULT_CONFIG.copy()
    config.update(data)
    return config, None

# Caso 2: archivo corrupto o formato inválido
except json.JSONDecodeError as e:
    logger.error("Configuración corrupta (%s): %s", path, e)
    return DEFAULT_CONFIG.copy(), (
        "El archivo de configuración está dañado o tiene un formato "
        "inválido. Se cargaron los valores por defecto."
    )

# Caso 3: falta de permisos de lectura
except PermissionError as e:
    logger.error("Sin permisos de lectura sobre %s: %s", path, e)
    return DEFAULT_CONFIG.copy(), (
        "No se tienen permisos para leer el archivo de configuración. "
        "Se cargaron los valores por defecto."
    )
```

Evidencia: el script tests/test_manual.py ejercita los 4 escenarios (archivo ausente, UTF-8 con tildes/ñ, archivo corrupto, y respaldo .bak) de forma automática.



The screenshot shows a code editor with the Explorer sidebar on the left, the Editor window in the center, and the Terminal window at the bottom. The Explorer sidebar shows a project structure with folders like 'ejemplos', 'src', and 'tests'. The Editor window displays the 'test_manual.py' script, which includes a docstring describing the manual verification script and its four test cases. The Terminal window shows the command 'python tests/test_manual.py' being executed, and the output displays the results of the four test cases, all of which passed successfully.

```
tests > test_manual.py
1 """
2 test_manual.py
3
4 Script de verificación manual (no requiere pytest) para evidenciar:
5 1. Codificación UTF-8: nombre_usuario e idioma con tildes/ñ sobreviven
6   un ciclo completo de guardado + carga.
7 2. Caso "archivo ausente": carga limpia con valores por defecto.
8 3. Caso "archivo corrupto": carga limpia con valores por defecto + aviso.
9 4. Escritura segura + respaldo: tras dos guardados consecutivos existe
10  un config.bak con el contenido del guardado anterior.
11
12 Ejecutar con: python tests/test_manual.py
13 """
14
15 import os
16 import sys
17
18 sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
```

```
(venv) arenz@MacBook-Pro-de-Arenz proyecto1-manejo-archivos % python -u "/Users/arenz/Library/CloudStorage/OneDrive-UniversidadRafaelLandivar/4TO. SEMESTRE/MANEJO E IMPLEMENTACIÓN DE ARCHIVOS/PROYECTOS/Proyecto 1/proyecto1-manejo-archivos/tests/test_manual.py"
[OK] Caso archivo ausente -> valores por defecto, sin excepción.
[OK] UTF-8: tildes y ñ se guardaron y recuperaron sin corrupción.
Configuración corrupta (/Users/arenz/Library/CloudStorage/OneDrive-UniversidadRafaelLandivar/4TO. SEMESTRE/MANEJO E IMPLEMENTACIÓN DE ARCHIVOS/PROYECTOS/Proyecto 1/proyecto1-manejo-archivos/tests/config_test.json): Expecting property name enclosed in double quotes: line 1 column 3 (char 2)
[OK] Caso archivo corrupto -> aviso mostrado: 'El archivo de configuración está dañado o tiene un formato inválido. Se cargaron los valores por defecto.'
[OK] Respaldo (.bak) conserva la configuración anterior al guardado.

Todas las verificaciones manuales pasaron correctamente.
(venv) arenz@MacBook-Pro-de-Arenz proyecto1-manejo-archivos %
```

Salida de tests/test_manual.py: los cuatro casos (archivo ausente, UTF-8, archivo corrupto, respaldo) muestran [OK] y el mensaje final confirma que todas las verificaciones pasaron correctamente.

4. Ejemplo de archivo de configuración generado

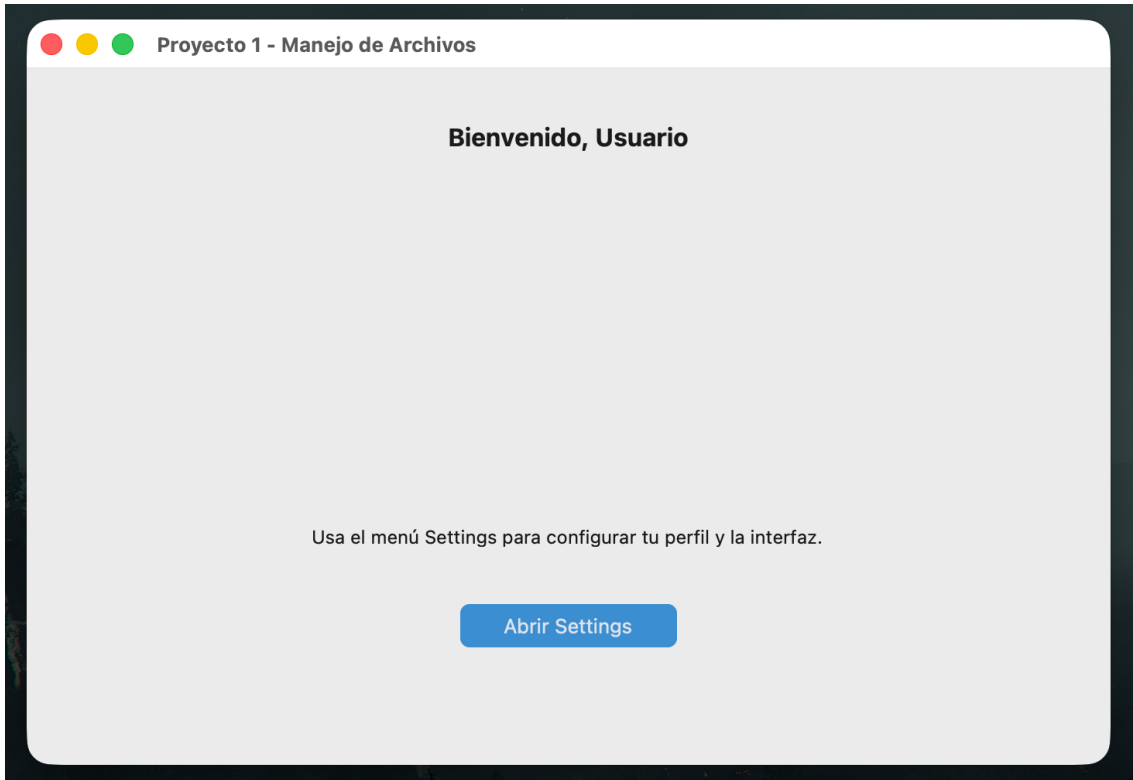
Toda lectura y escritura especifica `encoding="utf-8"` explícitamente, y `json.dump` se llama con `ensure_ascii=False` para que tildes y la letra ñ se guarden como caracteres reales. Ejemplo generado por la propia aplicación (`ejemplos/config.ejemplo.json`):

```
ejemplos > {} config.ejemplo.json > ...
1  {
2      "nombre_usuario": "María José Peláez Virula",
3      "tema_interfaz": "oscuro",
4      "idioma": "es-ES",
5      "tamaño_fuente": 16,
6      "color_barra_menu": "#1F2937",
7      "color_letra": "#F5F5F5",
8      "foto_perfil": "/Users/arenz/Pictures/perfil.png"
9  }
```

El nombre de usuario "María José Peláez Virula" conserva tildes y mayúsculas acentuadas correctamente tras un ciclo completo de guardado y carga, evidenciando el manejo correcto de codificación UTF-8. El archivo `ejemplos/config.ejemplo.json.bak` del repositorio contiene la versión previa a este guardado, como evidencia del mecanismo de respaldo.

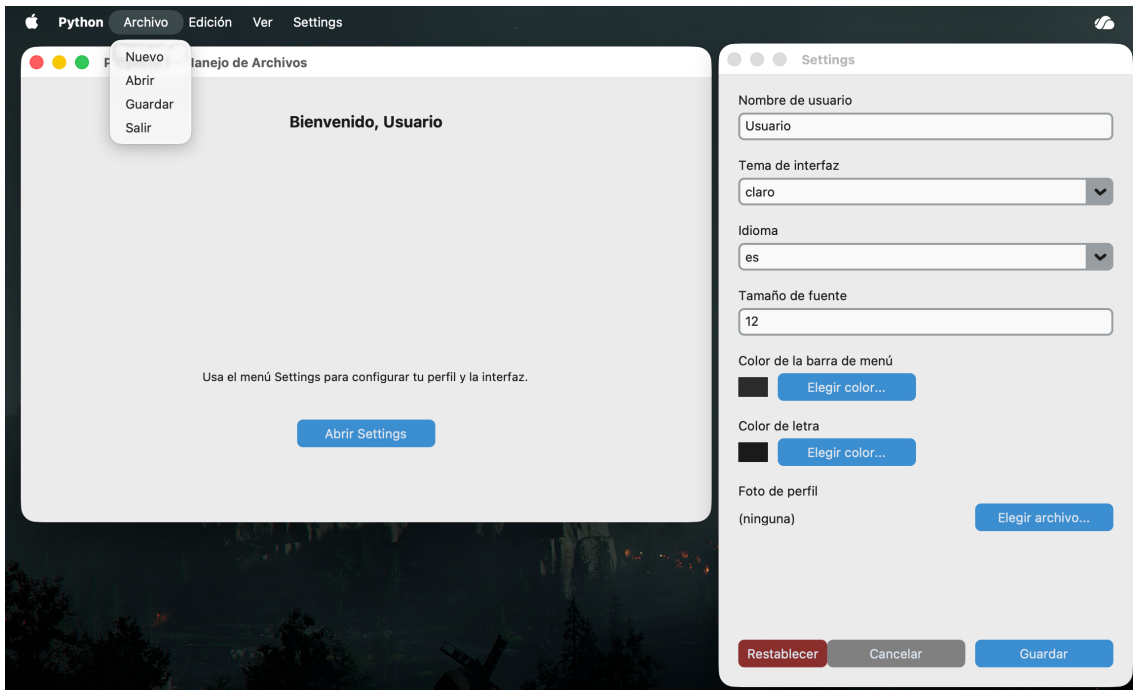
5. Capturas de pantalla del funcionamiento

5.1. Carga inicial (archivo ausente → valores por defecto)



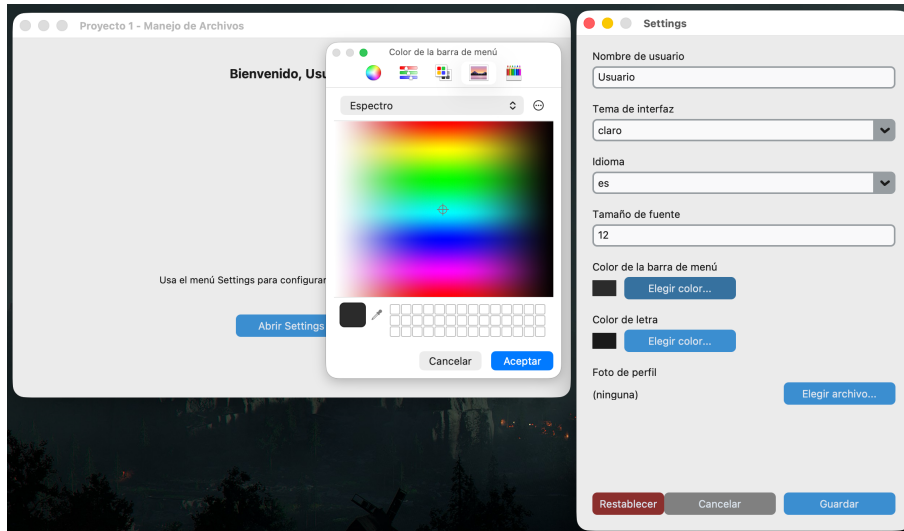
Primer arranque sin config.json: la aplicación carga los valores por defecto sin lanzar excepción.

5.2. Menú principal con Settings funcional

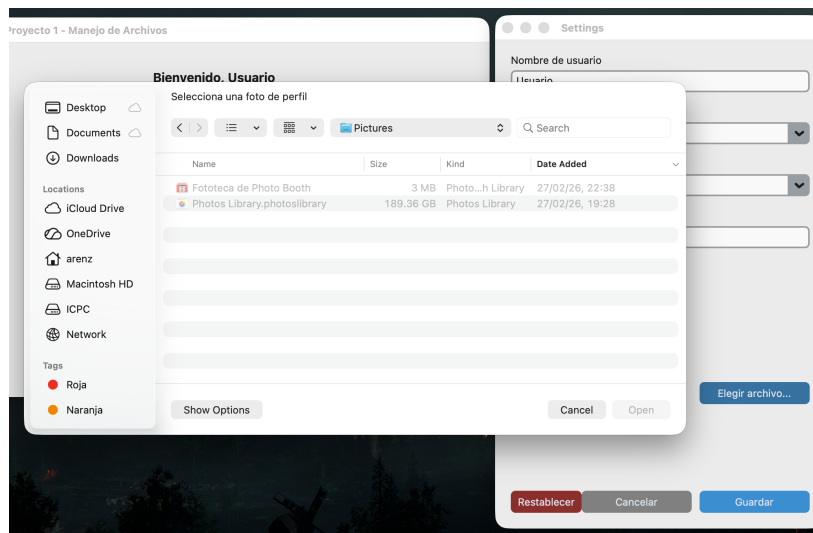


Menú Archivo desplegado (simulado) y la opción Settings visible en la barra, junto a Edición y Ver.

5.3. Selectores nativos: color y foto de perfil

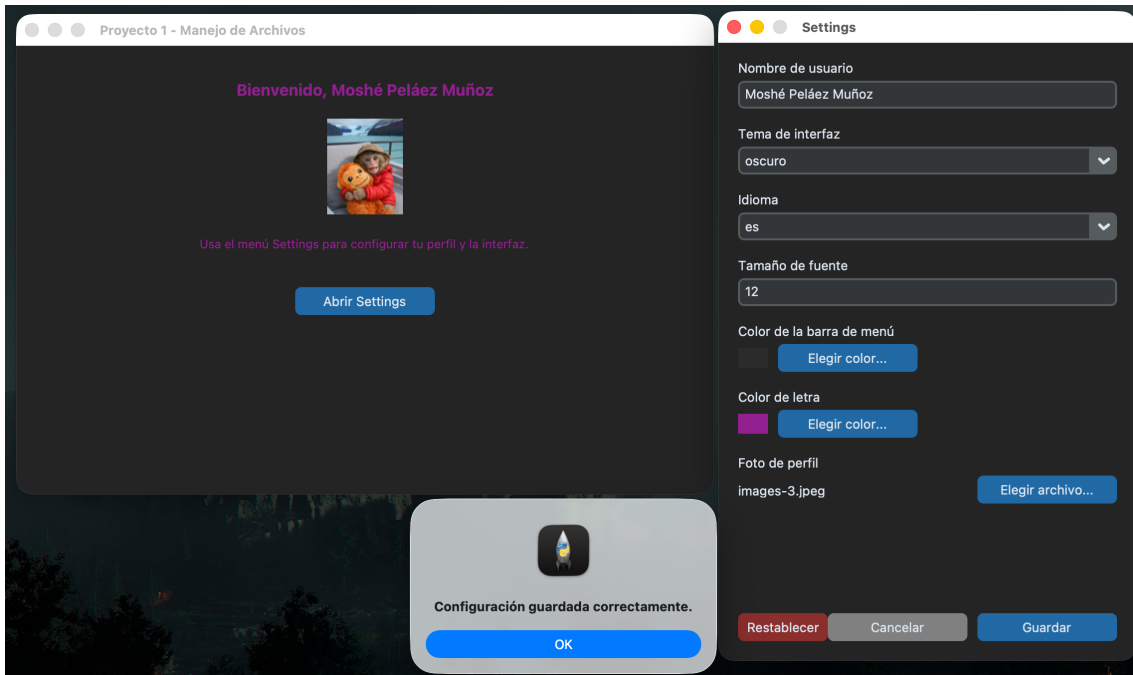


Selector de color nativo del sistema operativo (tkinter.colorchooser) para la barra de menú.



Explorador de archivos nativo (tkinter.filedialog) para elegir la foto de perfil.

5.4. Formulario de Settings completo y guardado exitoso



Formulario de Settings con nombre de usuario con tildes/ñ, tema de interfaz, color y foto seleccionados